

JAVA Beans



JavaBeans are reusable software components for Java. It is a java class that is developed by following some standards.

Java beans does not contains any logics. It acts as a helper class to carry data from one class to another class or from one project to another project.

In order to pass more than 3 vales from one class to another, take support of Java Bean.

The standards to develop a java bean class are following:-

1. Class must be taken as public.
2. It is recommended to implement `java.io.Serializable` interface to send data over the network.



3. All bean properties (Member variable) must be taken as private and non static.
4. Every bean property should have one public setter method and one getter method(Accessor method). Here, setter method will be used to set the values of the property(member variable) and getter method will be used to get those values from variables.
5. There must be one zero parameter constructor explicitly(given by programmer) or implicitly(given by java compiler as default constructor).



Advantages of JavaBeans:

1. Reusability:

JavaBeans are designed for reusability across different applications and projects, reducing code duplication and development time.

1. Modularity and Encapsulation:

They promote modularity by encapsulating data and behavior within a single component, making code easier to understand and maintain.

2. Introspection:

The JavaBeans specification allows tools to automatically analyze a bean's properties, methods, and events, facilitating visual development and integration with IDEs.

3. Serialization:

JavaBeans can be easily serialized and deserialized, allowing their state to be saved and restored.

4. Platform Independence:

Being written in Java, JavaBeans are inherently platform-independent and can run on any system with a Java Virtual Machine (JVM).

5. Integration with Frameworks:

JavaBeans seamlessly integrate with various Java frameworks like Spring and JSF, serving as fundamental building blocks for application development.

```
package com.bean;  
import java.io.*;  
public class StudentBean implements Serializable  
{
```

```
    //Member variable  
    private int roll;  
    private int id;  
    private String name;  
    private String course;
```

```
    //setter method to set the name  
    public void setName(String name)  
    {  
        this.name=name;  
    }
```

```
    //getter method to use the value  
    public String getName()  
    {  
        return name;  
    }
```

```
public void setCourse(String course)
{
    this.course=course;
}
public String getCourse()
{
    return course;
}

public void setRoll(int roll)
{
    this.roll=roll;
}
public int getRoll()
{
    return roll;
}

public void setID(int id)
{
    this.id=id;
}
public int getID()
{
    return id;
}
}
```



```
package com.bean;
public class BeanTest
{
    public static void main(String []str)
    {
        StudentBean student=new StudentBean();
        student.setID(101);
        student.setRoll(10);
        student.setName("Akshat");
        student.setCourse("MBBS");
        System.out.println("Student ID is "
+student.getID());
        System.out.println("Student Roll is "
+student.getRoll());
        System.out.println("Student Name is "
+student.getName());
        System.out.println("Student Course is "
+student.getCourse());
    }
}
```


JAVA Beans-Introspection

JavaBeans Introspection refers to the process by which external tools or frameworks analyze a JavaBean class to discover its properties, methods, and events at runtime.

This is a key feature of the JavaBeans API, enabling tools and applications (like IDEs or GUI builders) to dynamically discover and interact with the internal characteristics of a JavaBean.

In simple words:

Introspection in JavaBeans is the mechanism used to find out what a Bean can do — such as which properties it has (getter/setter methods), what events it can generate, and which methods it exposes — without knowing its internal implementation.

Key Concepts of Introspection

1. Properties:

- Attributes of a JavaBean that can be read or written.
- Follow getter and setter naming conventions:
 - `getPropertyName()` for reading.
 - `setPropertyName(value)` for writing.
 - For boolean properties, `isPropertyName()` can be used.

2. Methods:

Public methods exposed by the JavaBean for interaction.

3. Events:

Mechanism to notify listeners about changes or actions.

How Introspection Works

Introspection relies on:

1. Java Reflection API:

- Dynamically inspects the class to identify methods and fields.

2. Naming Conventions:

- Follows standard getter and setter patterns to identify properties.

3. `java.beans.Introspector` Class:

- Provides methods to analyze a JavaBean and retrieve its `BeanInfo`.

Example: Using Introspection

```
package com.beanIntrospect;
import java.io.*;
import java.beans.BeanInfo;
import java.beans.Introspector;
import java.beans.IntrospectionException;
import java.beans.PropertyDescriptor; import
java.beans.MethodDescriptor;
import java.beans.EventSetDescriptor;
public class IntrospectStudentBean implements Serializable
{
    //Member variable
    private int roll;
    private int id;
    private String name;
    private String course;

    //setter method to set the name
    public void setName(String name)
    {
        this.name=name;
    }
    //getter method to use the value
    public String getName()
    {
        return name; }
}
```



```
public void setCourse(String course)
```

```
{
```

```
    this.course=course;
```

```
}
```

```
public String getCourse()
```

```
{
```

```
    return course;
```

```
}
```

```
public void setRoll(int roll)
```

```
{
```

```
    this.roll=roll;
```

```
}
```

```
public int getRoll()
```

```
{
```

```
    return roll;
```

```
}
```

```
public void setID(int id)
```

```
{
```

```
    this.id=id;
```

```
}
```

```
public int getID()
```

```
{
```

```
    return id;
```

```
}
```

```
public static void main(String[] args)
{
    try
    {
        BeanInfo beanInfo = Introspector.getBeanInfo(IntrospectStudentBean.class);

        // Display Properties
        System.out.println("----Properties of MyBean:----");
        for (PropertyDescriptor pd : beanInfo.getPropertyDescriptors())
        {
            System.out.println(" - " + pd.getName() + " (Type: " +
pd.getPropertyType().getName() + ")");
        }

        // Display Methods
        System.out.println("---- Methods of MyBean ----");
        MethodDescriptor[] methods = beanInfo.getMethodDescriptors();
        for (MethodDescriptor method : methods) {
            System.out.println(method.getName());
        }

        // Display Events
        System.out.println("\n---- Events of MyBean ----");
        EventSetDescriptor[] events = beanInfo.getEventSetDescriptors();
        for (EventSetDescriptor event : events) {
            System.out.println(event.getName());
        }
    }
    catch (IntrospectionException e)
    {
        e.printStackTrace();
    }
}
```

Bound properties

In JavaBeans, **bound properties** are properties that notify registered listeners whenever their value changes i.e fire an event when their value changes. They are typically exposed through get and set methods.

This is useful for implementing event-driven programming, where changes in one component can automatically update other components.

Key points of Bound Properties:

1. **Property Change Notification:** When the value of a bound property changes, a **PropertyChangeEvent** is fired. It includes:
 - **source:** The object (bean) that fired the event.
 - **propertyName:** The programmatic name of the property that changed.
 - **oldValue:** The value of the property before the change.
 - **newValue:** The value of the property after the change.



2. **PropertyChangeListener:** This is an interface that defines a single method, `propertyChange(PropertyChangeEvent evt)`.

Objects interested in property changes implement the `PropertyChangeListener` interface and register themselves with the bean.

3. **PropertyChangeSupport Class:** This is a helper class that simplifies the management of `PropertyChangeListener` registrations and the firing of `PropertyChangeEvent`s.

Example: JavaBean with a bound property

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class Person
{
    private String name;
    private final PropertyChangeSupport support;

    // Constructor
    public Person()
    {
        support = new PropertyChangeSupport(this);
    }

    // Getter for the 'name' property
    public String getName()
    {
        return name;
    }
}
```



```
// Setter for the 'name' property (bound property)
public void setName(String newName)
{
    String oldName = this.name;
    this.name = newName;

    // Notify listeners about the property change
    support.firePropertyChange("name", oldName, newName);
}

// Add a listener
public void
addPropertyChangeListener(PropertyChangeListener listener) {
    support.addPropertyChangeListener(listener);
}

// Remove a listener
public void
removePropertyChangeListener(PropertyChangeListener
listener) {
    support.removePropertyChangeListener(listener);
}
}
```

```
import java.beans.PropertyChangeListener;

public class Main
{
    public static void main(String[] args)
    {
        Person person = new Person();

        // Add a listener to monitor changes to the 'name' property
        person.addPropertyChangeListener(new
PropertyChangeListener() {
            @Override
            public void propertyChange(PropertyChangeEvent evt)
            {
                System.out.println("Property '" + evt.getPropertyName() +
"" changed:");
                System.out.println("Old Value: " + evt.getOldValue());
                System.out.println("New Value: " + evt.getNewValue());
            }
        });

        // Change the 'name' property
        person.setName("Aman");
        person.setName("Akshat");
    }
}
```

Constrained Properties - JavaBeans

A **constrained property** in JavaBeans is a property whose changes can be vetoed by registered listeners. This mechanism allows external objects to monitor and potentially reject changes to a property, ensuring that the property value adheres to specific rules or conditions.

Key points of Constrained Properties:

1. **VetoableChangeListener:**
 - A listener interface that allows objects to veto property changes.
 - It has a single method:
`vetoableChange(PropertyChangeEvent evt)`.
2. **PropertyChangeEvent:**
 - Encapsulates information about a property change, such as the property name, old value, and new value.
3. **PropertyChangeSupport and VetoableChangeSupport:**
 - Helper classes to manage listeners and fire property change events.



Steps to Create a Constrained Property:

- I. Use `VetoableChangeSupport` to manage vetoable listeners.
- II. Fire a vetoable change event before updating the property.
- III. Allow listeners to veto the change by throwing a `PropertyVetoException`.

Example: Constrained Property in a JavaBean

```
import java.beans.*;
import java.io.Serializable;

public class ConstrainedBean implements Serializable
{
    private String name;
    private final VetoableChangeSupport vetoableChangeSupport;

    // Constructor
    public ConstrainedBean()
    {
        vetoableChangeSupport = new VetoableChangeSupport(this);
    }

    // Getter for 'name'
    public String getName()
    {
        return name;
    }

    // Setter for 'name' with constrained property logic
    public void setName(String newName) throws PropertyVetoException
    {
        String oldName = this.name;
        // Fire vetoable change event
        vetoableChangeSupport.fireVetoableChange("name", oldName, newName);

        // If no exception is thrown, update the property
        this.name = newName;
    }
}
```



```
// Add a VetoableChangeListener
public void addVetoableChangeListener(VetoableChangeListener listener)
{
    vetoableChangeSupport.addVetoableChangeListener(listener);
}

// Remove a VetoableChangeListener
public void removeVetoableChangeListener(VetoableChangeListener listener)
{
    vetoableChangeSupport.removeVetoableChangeListener(listener);
}
}
```



```
import java.beans.PropertyVetoException;

public class Main {
    public static void main(String[] args) {
        ConstrainedBean bean = new ConstrainedBean();

        // Add a vetoable change listener
        bean.addVetoableChangeListener(evt -> {
            String newValue = (String) evt.getNewValue();
            if (newValue == null || newValue.isEmpty()) {
                throw new PropertyVetoException("Name cannot be
empty!", evt);
            }
        });

        try {
            bean.setName("John Doe"); // Valid change
            System.out.println("Name set to: " + bean.getName());

            bean.setName(""); // Invalid change, will throw exception
        } catch (PropertyVetoException e) {
            System.out.println("Change vetoed: " + e.getMessage());
        }
    }
}
```

Bean Persistence

- Java bean has the property of persistence when its properties, fields, and state information are saved to and retrieved from storage. Component models provide a mechanism for persistence that enables the state of components to be stored in a non-volatile place for later retrieval.
- The mechanism that makes persistence possible is called serialization. Object serialization means converting an object into a data stream and writing it to storage. Any applet, application, or tool that uses that bean can then "reconstitute" it by deserialization. The object is then restored to its original state.
- For example, a Java application can serialize a Frame window on a Microsoft Windows machine, the serialized file can be sent with e-mail to a Solaris machine, and then a Java application can restore the Frame window to the exact state which existed on the Microsoft Windows machine.



- Any applet, application, or tool that uses that bean can then "reconstitute" it by deserialization.
- All beans must persist. To persist, your beans must support serialization by implementing either the `java.io.Serializable` (in the API reference documentation) interface, or the `java.io.Externalizable` (in the API reference documentation) interface.
- These interfaces offer you the choices of automatic serialization and customized serialization. If any class in a class's inheritance hierarchy implements `Serializable` or `Externalizable`, then that class is serializable.

```
import java.io.Serializable;
```

```
public class EmployeeBean implements Serializable {
```

```
    private String name;
```

```
    private int age;
```

```
    public EmployeeBean() {} // No-arg constructor (required for JavaBean)
```

```
    public String getName()
```

```
    {
```

```
        return name;
```

```
    }
```

```
    public void setName(String name)
```

```
    {
```

```
        this.name = name;
```

```
    }
```

```
    public int getAge()
```

```
    {
```

```
        return age;
```

```
    }
```

```
    public void setAge(int age)
```

```
    {
```

```
        this.age = age;
```

```
    }
```

```
    @Override
```

```
    public String toString()
```

```
    {
```

```
        return "EmployeeBean{name='" + name + "', age='" + age + "'}";
```

```
    }
```

```
}
```



```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;

public class SaveBean {
    public static void main(String[] args) {
        try {
            EmployeeBean emp = new EmployeeBean();
            emp.setName("Abhishek Singh");
            emp.setAge(30);

            ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("employee.ser"));
            out.writeObject(emp);
            out.close();

            System.out.println("Bean state saved successfully!");
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}
```




```
import java.io.FileInputStream;
import java.io.ObjectInputStream;

public class LoadBean {
    public static void main(String[] args) {
        try
        {
            ObjectInputStream in = new ObjectInputStream(new
FileInputStream("employee.ser"));
            EmployeeBean emp = (EmployeeBean) in.readObject();
            in.close();

            System.out.println("Loaded Bean: " + emp);
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

Customizers in Java Bean

- Customizers in Java Bean are classes that allow developers to modify the appearance and behavior of an object at design time.
- They provide a graphical user interface (GUI) that allows developers to modify an object's properties and view the changes in real-time.
- They are an essential tool for creating dynamic user interfaces in Java Beans.

How do Customizers in Java Bean work?

Customizers in Java Bean work by providing a GUI that allows developers to modify an object's properties. They are implemented as classes that extend the `java.beans.PropertyEditorSupport` class.

This class provides methods that allow developers to modify an object's properties and view the changes in real-time.

To use a Customizer in Java Bean, you must first register it with the Java Bean infrastructure. Once registered, the Customizer can be invoked from within an integrated development environment (IDE) such as BDK, Eclipse or NetBeans. The Customizer will then appear as a dialog box that allows developers to modify an object's properties.



Benefits of Customizers in Java Bean

Customizers in Java Bean offer several benefits to software developers, including:

- **Dynamic user interfaces:** It allows developers to create dynamic user interfaces that respond to user input in real-time.
- **Simplified development:** It can simplify development by providing a GUI for modifying an object's properties, reducing the amount of code that developers need to write.
- **Improved productivity:** It can improve productivity by allowing developers to modify an object's properties without having to recompile and redeploy the application.
- **Better user experience:** It can improve the user experience by providing a graphical interface that is easy to use and understand.

How Customizers Work

- **Implementation:** A customizer class must implement the `java.beans.Customizer` interface. This interface defines methods for setting the object to be customized and for listening to property changes.
- **GUI Integration:** Customizers often extend `java.beans.PropertyEditorSupport` and provide a **custom GUI panel** where developers can interactively change bean properties.
- **Design-Time Use:** These panels are displayed in IDEs or builder tools, allowing developers to see and modify bean behavior in real time.
- **Event Handling:** Customizers notify the bean of changes using `PropertyChangeListener`, ensuring the bean updates its internal state accordingly.

Example Workflow

- Create a bean class with properties.
- Develop a customizer class that implements Customizer.
- Register the customizer in the bean's BeanInfo class.
- Use a builder tool to load the bean and open the customizer panel.



Example: Java Bean with a Customizer

Let's say we have a bean called `ColorBean` that stores a color.

We want to create a customizer that lets users pick a color using a *GUI*.

The Bean Class

```
public class ColorBean implements java.io.Serializable
{
    private Color color = Color.WHITE;

    public Color getColor()
    {
        return color;
    }

    public void setColor(Color c)
    {
        color = c;
    }
}
```

The Customizer Class

```
public class ColorBeanCustomizer extends JPanel implements
Customizer {
    private ColorBean bean;

    private PropertyChangeSupport pcs = new
PropertyChangeSupport(this);

    public ColorBeanCustomizer() {
        JButton colorButton = new JButton("Choose Color");

        colorButton.addActionListener(e -> {
            Color newColor = JColorChooser.showDialog(this, "Select
Color", bean.getColor());

            if (newColor != null) {
                Color oldColor = bean.getColor();
                bean.setColor(newColor);
                pcs.firePropertyChange("color", oldColor, newColor);
            }
        });
        add(colorButton);
    }
}
```



@Override

```
public void setObject(Object bean) {  
    this.bean = (ColorBean) bean;  
}
```

@Override

```
public void addPropertyChangeListener(PropertyChangeListener  
listener) {  
    pcs.addPropertyChangeListener(listener);  
}
```

@Override

```
public void  
removePropertyChangeListener(PropertyChangeListener listener) {  
    pcs.removePropertyChangeListener(listener);  
}  
}
```

Registering the Customizer in BeanInfo

```
public class ColorBeanBeanInfo extends SimpleBeanInfo
{
    @Override
    public Class<?> getBeanDescriptor() {
        return ColorBean.class;
    }

    @Override
    public BeanDescriptor getBeanDescriptor() {
        return new BeanDescriptor(ColorBean.class,
ColorBeanCustomizer.class);
    }
}
```

Process Flow

- The ColorBeanCustomizer provides a button to open a color chooser.
- When the user picks a color, it updates the bean and fires a property change event.
- The builder tool (like NetBeans) listens for these events and updates the design-time view accordingly.

Benefits of Using Java Beans API

There are several benefits to using the Java Beans API in software development. Some of these benefits include:

Reusability: Java Beans are designed to be reusable, which means that they can be used in multiple applications without having to rewrite the code.

Modularity: Java Beans are modular, which means that they can be easily assembled to create complex applications.

Simplified Development: Java Beans provide a simple and consistent interface for interacting with other parts of the application.

Interoperability: Java Beans are designed to be interoperable, which means that they can be used in applications developed using different programming languages.

Usage of Java Beans API

The Java Beans API is used extensively in enterprise applications and other software development projects. Some of the common uses of Java Beans API include:

- **Graphical User Interfaces:** Java Beans are often used to create graphical user interfaces for desktop and web applications.
- **Business Logic:** Java Beans are used to implement the business logic of applications, such as validation and data processing.
- **Persistence:** Java Beans are often used to implement persistence, allowing data to be stored and retrieved from databases or files.
- **Web Services:** Java Beans are used to implement web services, which allow applications to communicate with each other over the internet.



Java Beans API

Java Beans API is a set of guidelines and conventions that define a standard for creating reusable software components in Java. It provides a framework for creating objects that are easily integrated into other applications.

Java Beans are a type of software component that can be used in any Java application. They are essentially modular, reusable software modules that can be easily assembled to create complex applications. Java Beans are designed to be self-contained, meaning that they have all the necessary functionality built into them, and can be easily manipulated by other applications.

Key Features of Java Beans API

The Java Beans API has several key features that make it an essential part of software development. Some of these features include:

- **Encapsulation:** Java Beans are designed to be encapsulated, which means that their implementation details are hidden from the outside world. This helps to prevent other parts of the application from directly accessing or modifying the internal state of the Java Bean.
- **Properties:** Java Beans have a set of properties that define their state. These properties can be read or modified by other parts of the application.
- **Events:** Java Beans can generate events when their properties change. Other parts of the application can listen for these events and respond accordingly.
- **Persistence:** Java Beans can be made persistent, which means that they can be saved to a database or file and retrieved at a later time.





